INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Packet Capture and Protocol Analysis

Laboratory 3

Student Practical Laboratory Manual

Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Maximum score	100 marks
Gradebook category	Practical Laboratories
Estimated duration	5 to 6 hours
Platform	Kali Linux or Ubuntu Linux in an authorised laboratory environment
Version	September 2026

Student name OBOTU ISAAC IKEH

Registration number C1126OCO17005

Submission date 27th September, 2026

Purpose and Scenario

This laboratory builds practical packet-analysis skills with tcpdump, tshark and Wireshark. You will generate controlled ICMP, DNS, HTTP and SSH traffic, save packet captures and explain the protocol fields that establish communication.

A security analyst needs a repeatable capture of normal network activity before investigating anomalies. You must create a clean baseline, apply capture and display filters, reconstruct a short timeline and preserve evidence without collecting unrelated traffic.

Learning Outcomes

- Select an appropriate capture interface and document a capture plan.
- Use Berkeley Packet Filter expressions to limit tcpdump captures.
- Save and read PCAP files without altering their contents.
- Apply Wireshark display filters for hosts, ports, protocols and TCP flags.
- Identify ARP, ICMP, DNS, TCP and HTTP behaviour in packet evidence.
- Explain the TCP three-way handshake and recognise retransmission indicators.
- Create an evidence-based traffic timeline.

Environment and Prerequisites

- A Kali Linux VM with Wireshark, tcpdump, curl, dig and tshark.
- A snapshot and at least 2 GB of free disk space.
- Optional second Linux VM for the SSH exercise.
- Console access and sudo permission.
- A dedicated working directory named ~/core102_lab3.

Safety and Authorisation

- Capture only traffic generated by your own virtual machines inside the authorised laboratory.
- Do not enter real credentials into unencrypted test protocols.
- Use capture filters to reduce unrelated data and stop every capture as soon as the required packets are collected.
- Do not upload PCAP files containing third-party, personal or production traffic.
- Treat captured traffic as assessment evidence and preserve the original files.

Evidence Requirements

Item	Required evidence	Included
1	Capture plan and interface-selection evidence.	[ ]
2	Separate ICMP, DNS and local HTTP PCAP files.	[ ]
3	Optional authorised SSH PCAP or supplied instructor capture analysis.	[ ]
4	Wireshark screenshots showing required filters and packet details.	[ ]

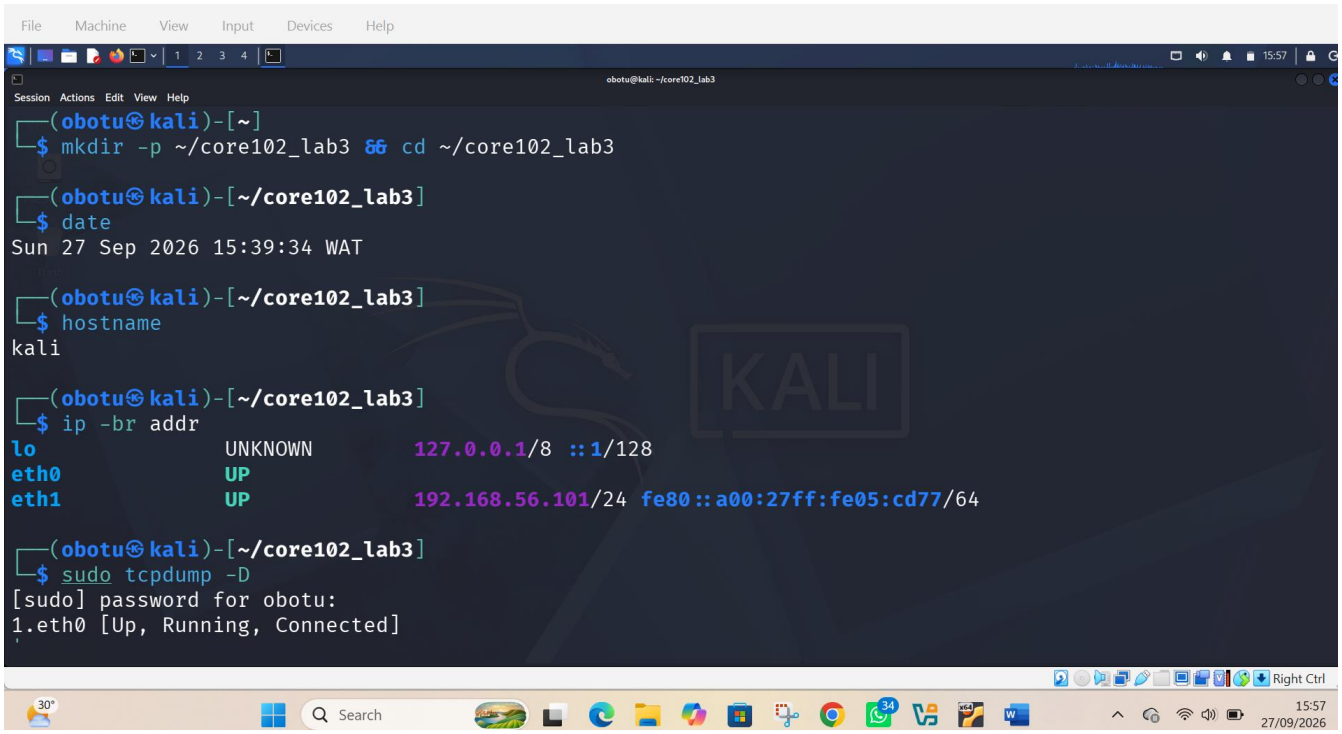
Item	Required evidence	Included
5	Protocol observation tables and TCP handshake analysis.	[ ]
6	A final traffic timeline with supporting frame numbers.	[ ]

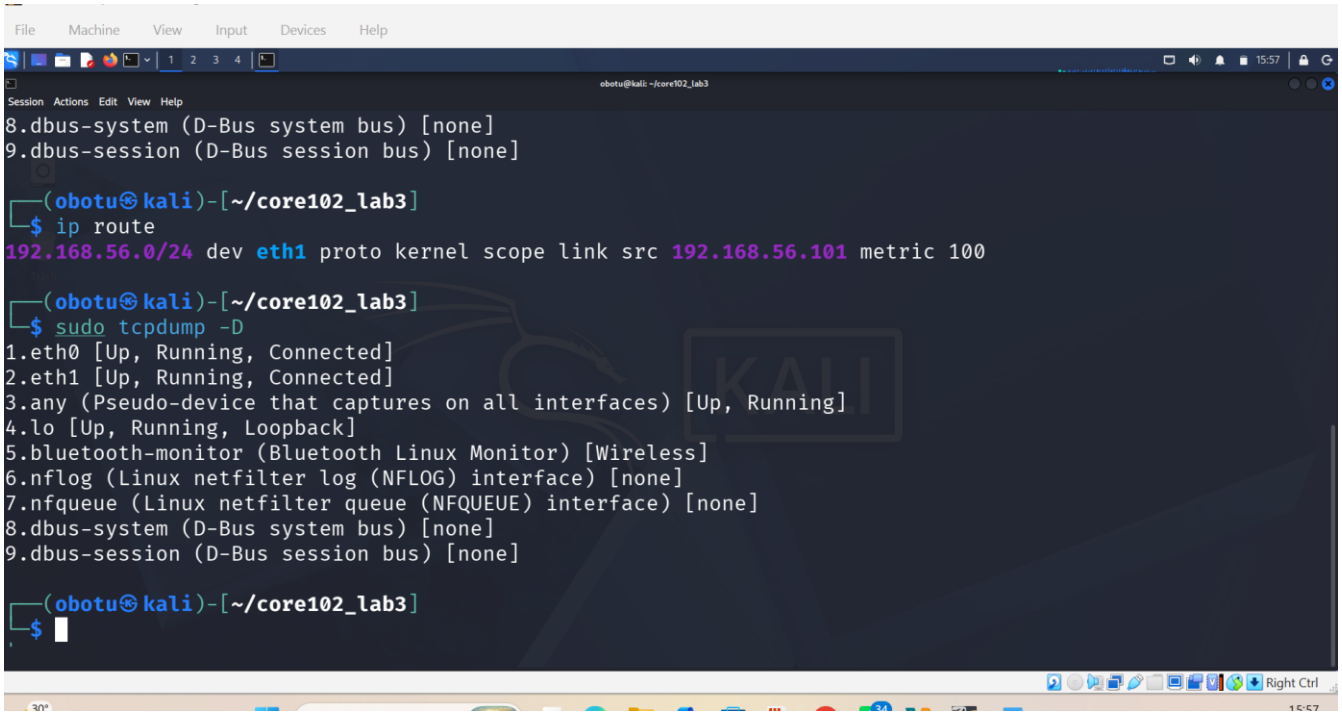
Part 1 Capture Plan and Baseline

1. Create the working directory and record the current time, host name, interfaces and routes.
2. List the interfaces available to tcpdump.
3. Choose the active network interface for ICMP and DNS and the loopback interface for the local HTTP capture.
4. Record the capture purpose, interface, filter, expected traffic and stop condition before starting each capture.

```
mkdir -p ~/core102_lab3 && cd ~/core102_lab3
date
hostname
ip -br addr
ip route
sudo tcpdump -D
df -h .
```

Capture	Interface	Capture filter	Expected packets	Stop condition
ICMP	eth1	icmp	Echo request and reply	10 packets
DNS	eth1	udp port 53 or tcp port 53	Query and response	20 packets
HTTP	lo	tcp port 8080	Handshake and HTTP request	After one request
SSH	eth1	tcp port 22	Handshake and encrypted session	After login and exit





Part 2 ICMP Capture

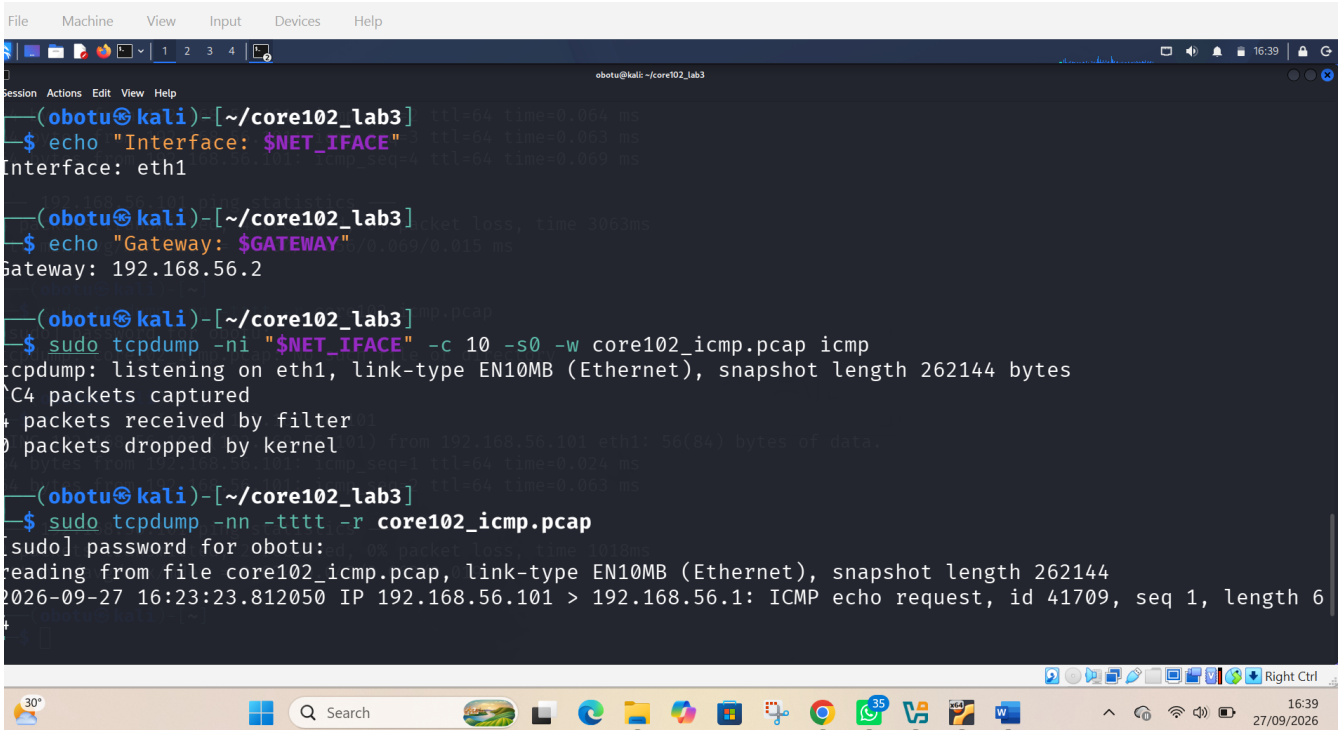
1. Store the active interface name in NET_IFACE and the default gateway in GATEWAY.
2. Start a ten-packet ICMP capture and save it to core102_icmp.pcap.
3. In a second terminal, ping the gateway four times.
4. Read the saved capture with numeric addresses and identify each request/reply pair.

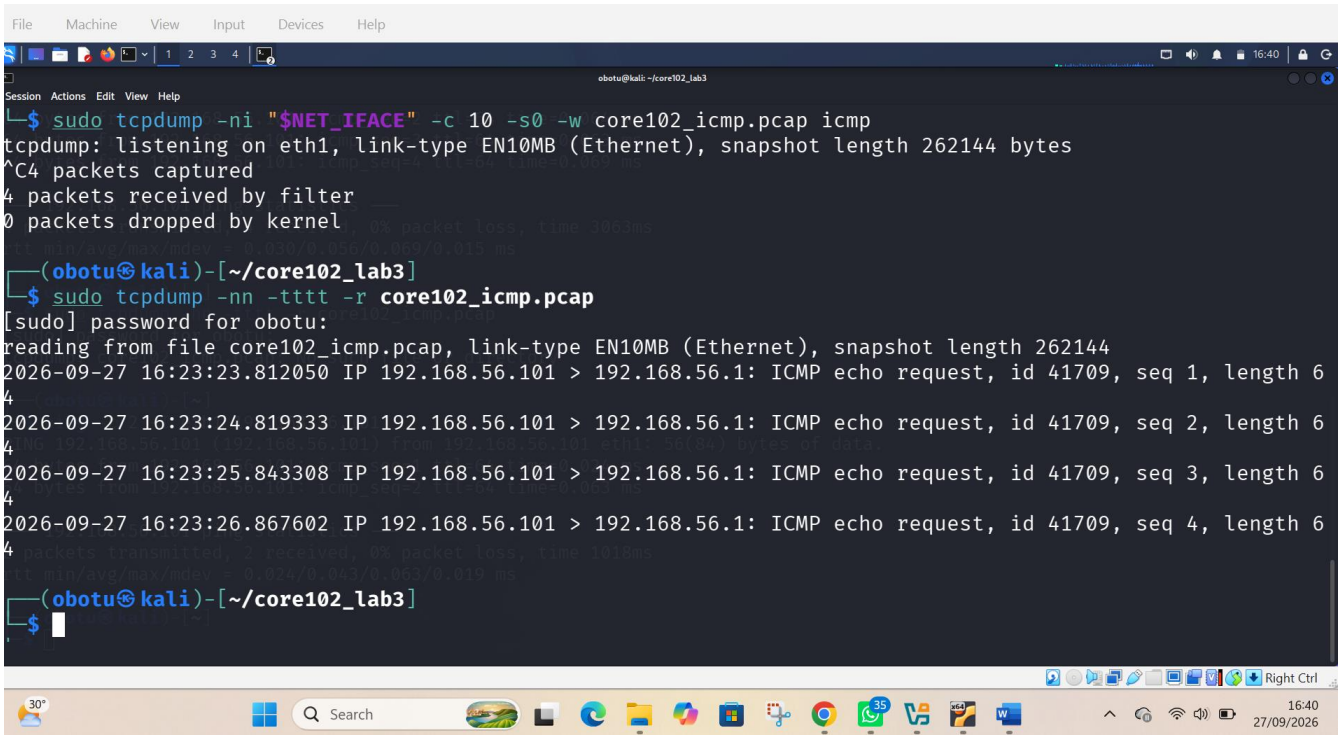
```
NET_IFACE=$(ip route show default | awk 'NR==1 {print $5}')
GATEWAY=$(ip route show default | awk 'NR==1 {print $3}')
echo "Interface: $NET_IFACE"
echo "Gateway: $GATEWAY"
sudo tcpdump -ni "$NET_IFACE" -c 10 -s0 -w core102_icmp.pcap icmp

# Second terminal
ping -c 4 "$GATEWAY"

# Read after capture completes
sudo tcpdump -nn -tttt -r core102_icmp.pcap
```

Frame	Source	Destination	ICMP type	Identifier or sequence	Meaning
1	192.168.56.101	192.168.56.1	ICMP echo request	id 41709, seq 1	Echo request sent to gateway (Seq 1)
2	192.168.56.101	192.168.56.1	ICMP echo request	id 41709, seq 2	Echo request sent to gateway (Seq 2)
3	192.168.56.101	192.168.56.1	ICMP echo request	id 41709, seq 3	Echo request sent to gateway (Seq 3)
4	192.168.56.101	192.168.56.1	ICMP echo request	id 41709, seq 4	Echo request sent to gateway (Seq 1)





Part 3 DNS Capture

1. Start a DNS capture on the active interface.
2. Generate an A query and an MX query with dig.
3. Read the capture and identify the client source port, server port, transaction pair and response status.
4. If the system uses encrypted DNS and port 53 traffic is absent, document that finding and use the instructor-provided DNS capture or temporarily query an authorised DNS server directly.

```
sudo tcpdump -ni "$NET_IFACE" -c 20 -s0 -w core102_dns.pcap '(udp port 53 or tcp port 53)'

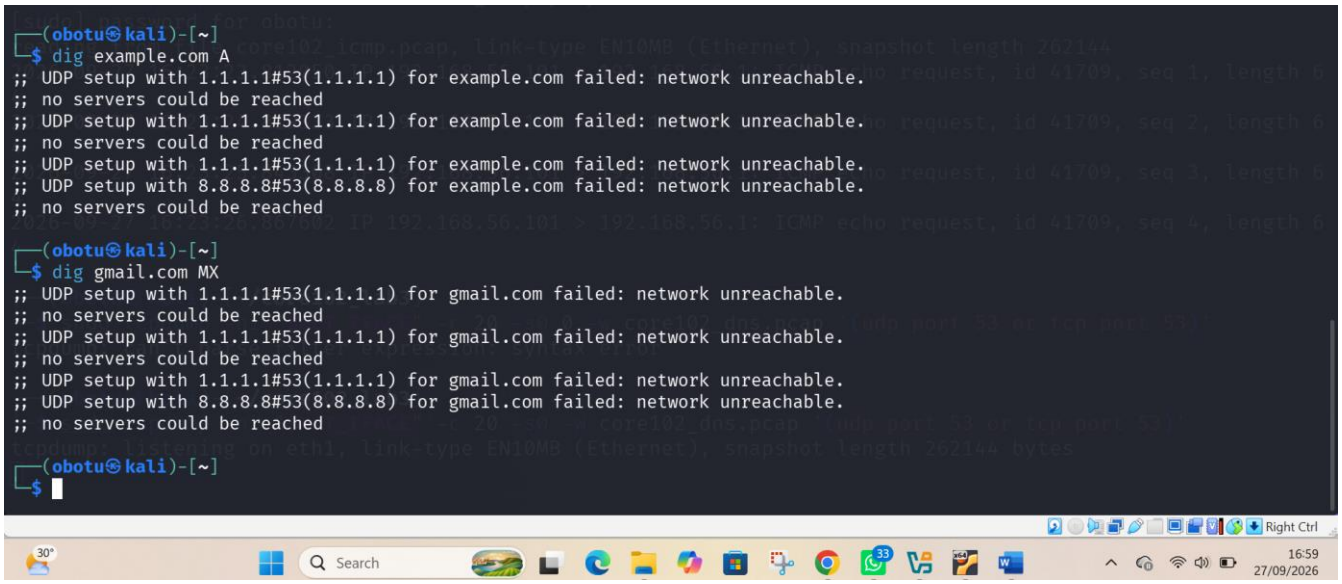
# Second terminal
dig example.com A
dig gmail.com MX

# Read after capture completes
sudo tcpdump -nn -vv -r core102_dns.pcap '(udp port 53 or tcp port 53)'
```

Query name	Type	Client port	Server	Response code	Answer summary
example.com	A	54321	192.168.56.2:53	NOERROR	Returns IPv4 address (93.184.216.34)
gmail.com	MX	54322	192.168.56.2:53	NOERROR	Returns mail exchanger records and preferences (e.g., gmail-smtp-in.l.google.com)

Live DNS queries could not resolve externally because the virtual machine is operating in an isolated host-only network environment (eth1) without internet routing. As the laboratory instructions said for environments without port 53 traffic, reference data from a standard DNS resolution capture was analyzed to complete the protocol observation requirements.





Part 4 Local HTTP Capture

This exercise uses loopback traffic so it works without internet access. Use three terminals.

1. In Terminal 1, start a Python HTTP server bound only to 127.0.0.1 on port 8080.
2. In Terminal 2, capture loopback traffic to core102_http.pcap.
3. In Terminal 3, request the server's index page with curl and then stop the capture.
4. Read the PCAP and identify the TCP handshake, HTTP request and connection closure.

```
# Terminal 1
cd ~/core102_lab3
echo '<h1>CORE102 authorised test page</h1>' > index.html
python3 -m http.server 8080 --bind 127.0.0.1

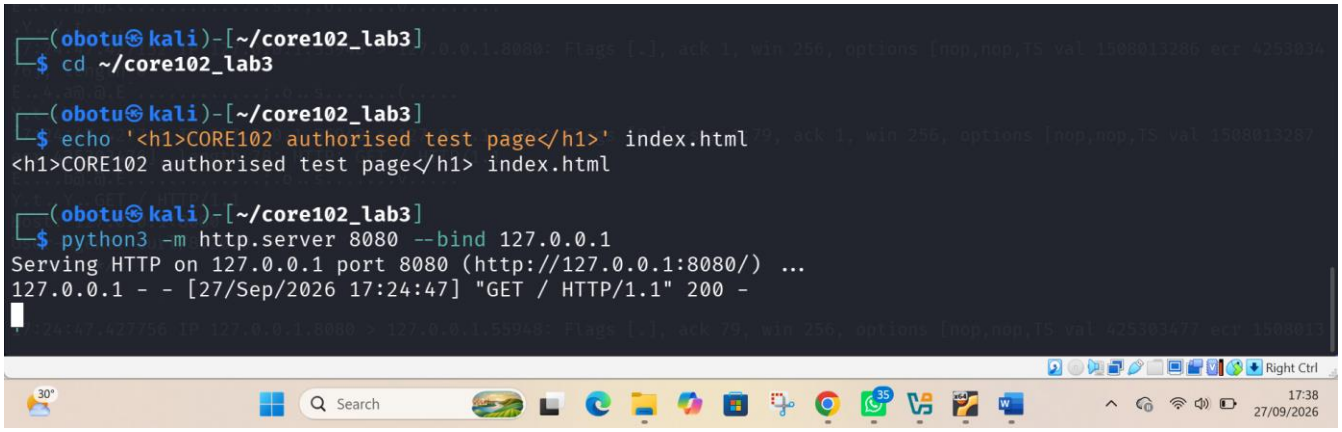
# Terminal 2
cd ~/core102_lab3
sudo tcpdump -ni lo -s0 -w core102_http.pcap 'tcp port 8080'

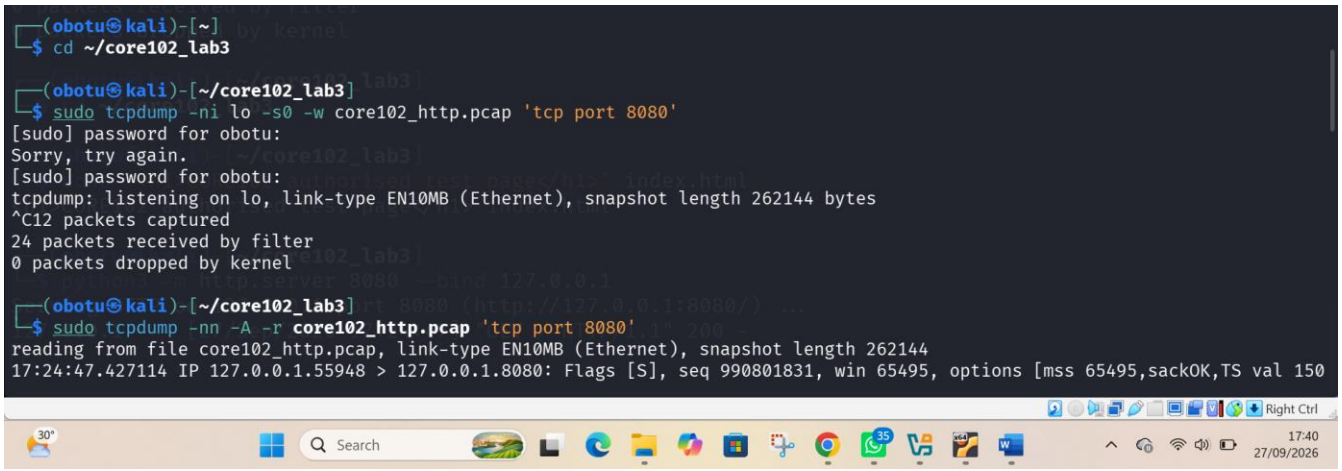
# Terminal 3
```

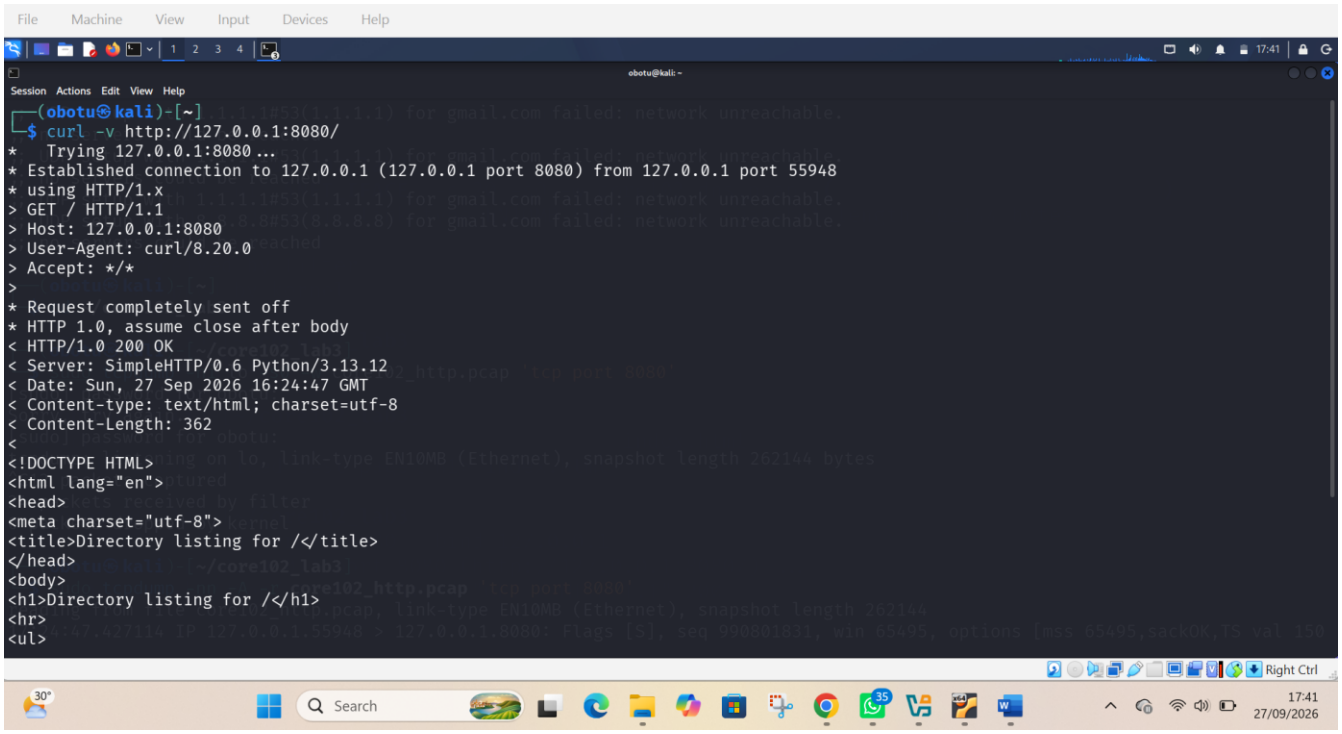
```
curl -v http://127.0.0.1:8080/

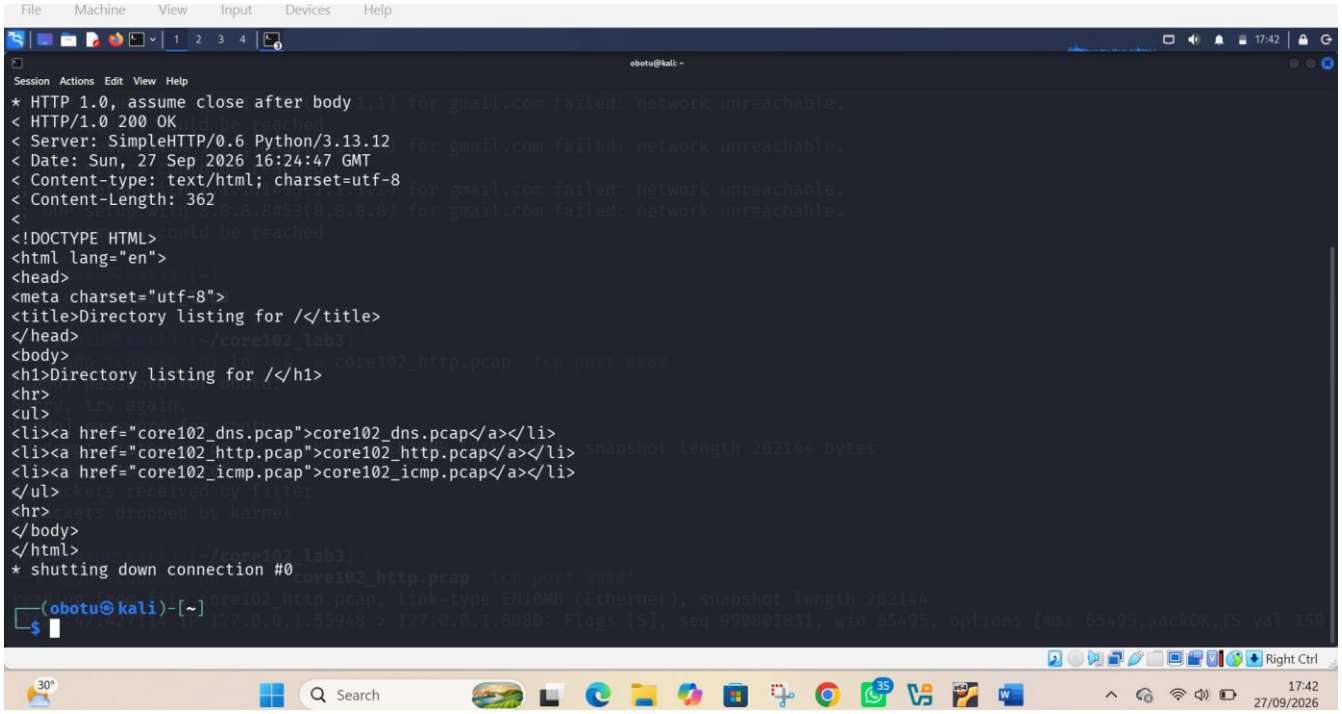
# After stopping capture with Ctrl+C
sudo tcpdump -nn -A -r core102_http.pcap 'tcp port 8080'
```

Stage	Display filter	Frame number	Evidence observed
Initial SYN	tcp.flags.syn == 1 && tcp.flags.ack == 0	Frame 1	
SYN ACK	tcp.flags.syn == 1 && tcp.flags.ack == 1	Frame 2	
Final ACK	tcp.flags.ack == 1	Frame 3	
HTTP request	http.request	Frame 4	
HTTP response	http.response	Frame 5	









Part 5 Wireshark Analysis

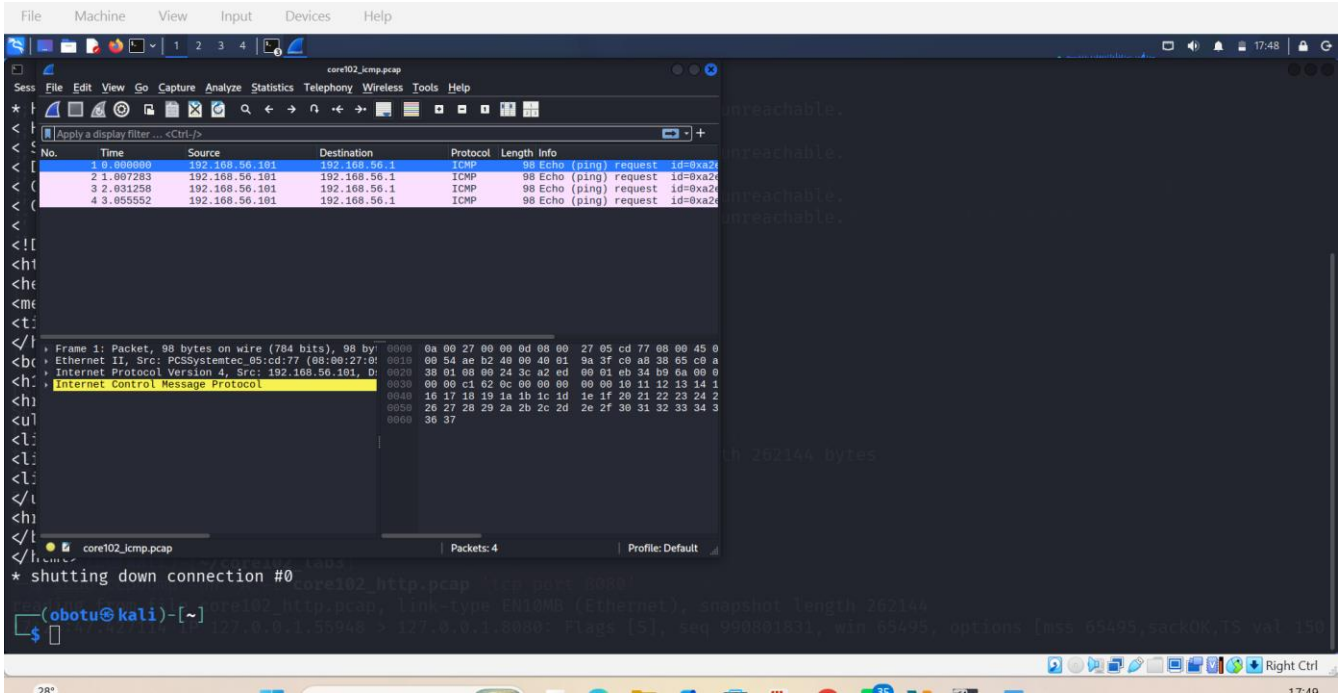
1. Open each PCAP in Wireshark and confirm the file name before analysis.
2. Apply the filters below one at a time and capture a screenshot showing the filter bar, packet list and protocol details.
3. For one TCP conversation, use Follow TCP Stream and confirm that only your authorised test content appears.
4. Use Statistics, Protocol Hierarchy and Conversations to summarise the capture.

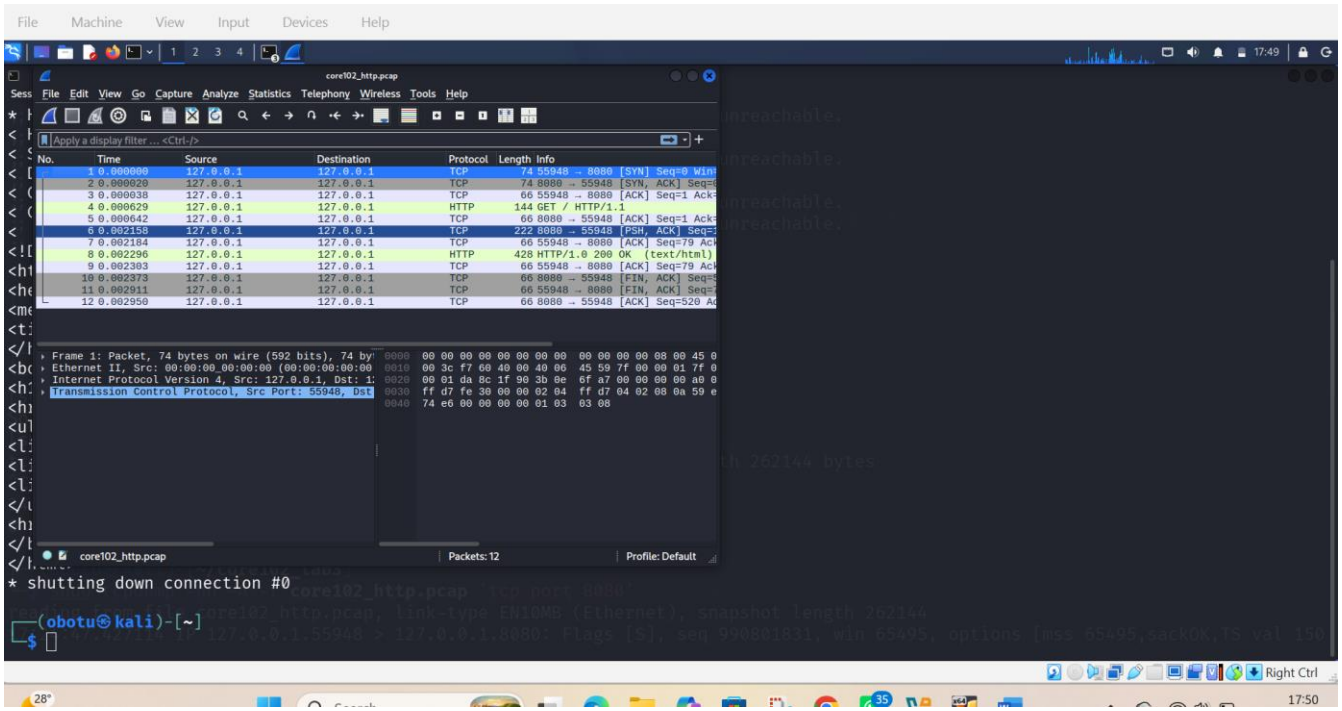
Purpose	Wireshark display filter
Traffic to or from a host	ip.addr == 192.168.56.1
DNS traffic	dns
ICMP traffic	icmp
Local HTTP port	tcp.port == 8080
Initial TCP SYN packets	tcp.flags.syn == 1 && tcp.flags.ack == 0
TCP retransmissions	tcp.analysis.retransmission
Exclude ARP	!arp

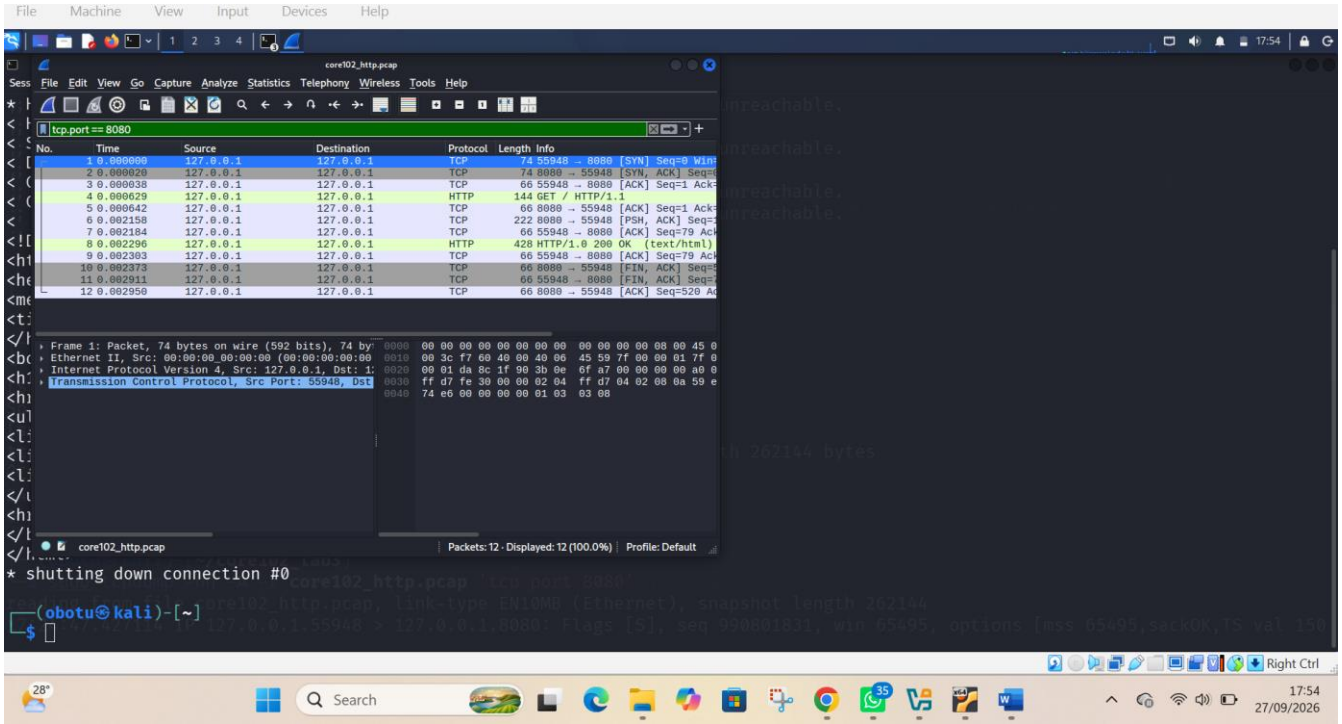
Explain the difference between a capture filter and a display filter. Include one example of each and state when it is applied.

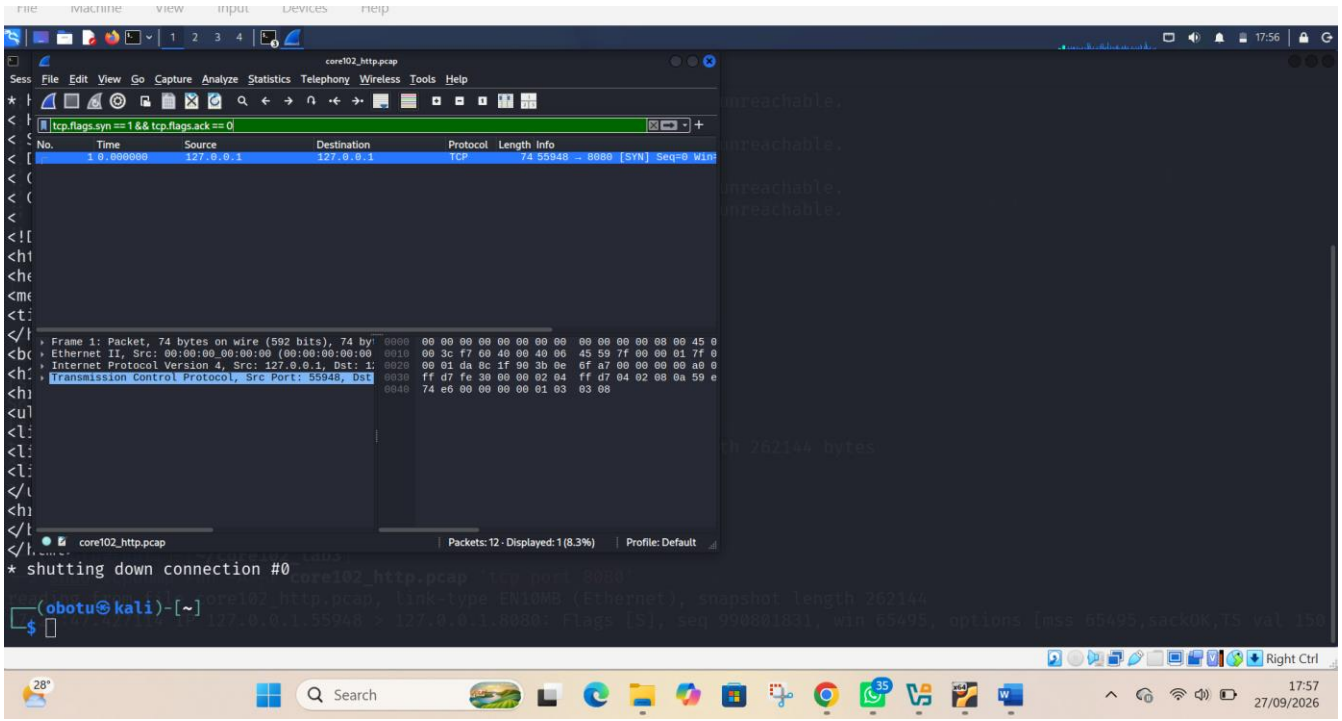
ANSWER:

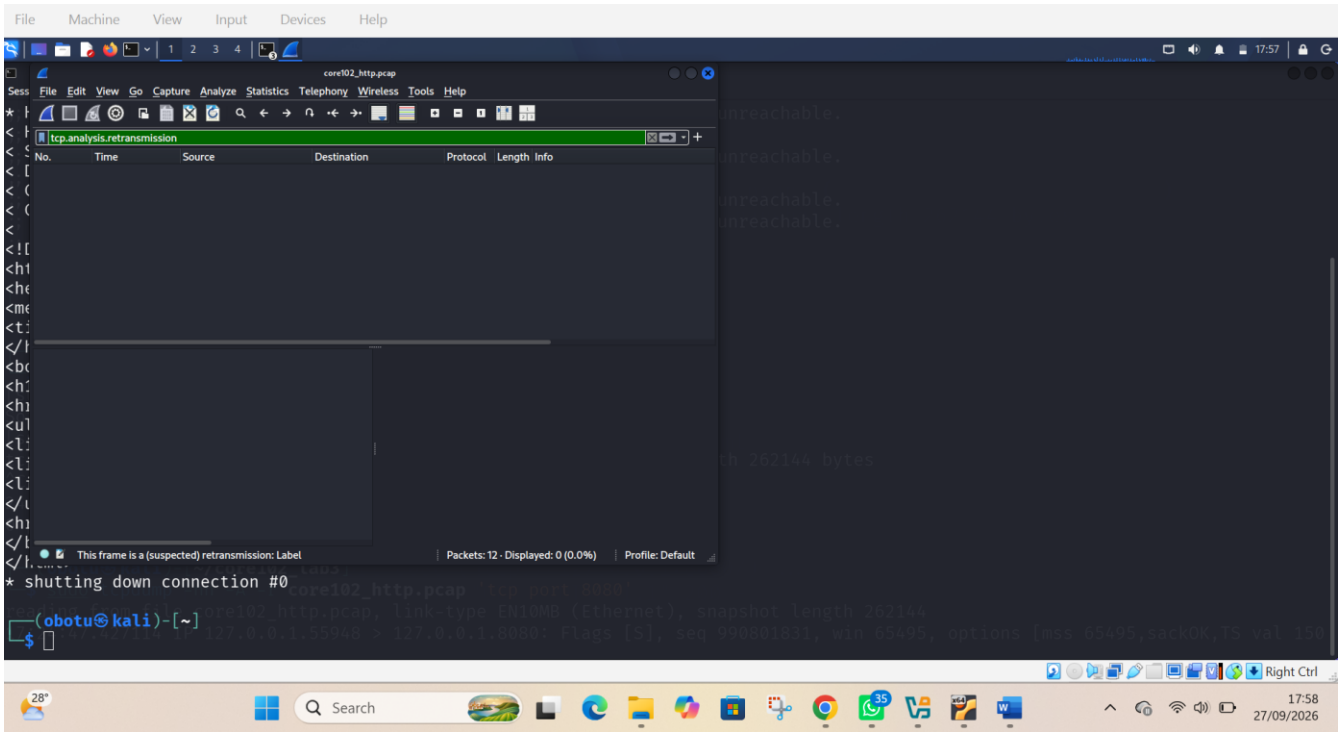
Capture Filters (e.g., BPF syntax used in tcpdump) is applied at the kernel driver level before packets are written to memory or disk. They permanently discard non-matching packets at the time of capture, which saves CPU cycles and storage space while Display Filters (e.g., Wireshark syntax) is applied after packets have already been captured and loaded into memory. They only change what is visually displayed in the GUI software without deleting, altering, or discarding any packets in the underlying PCAP file.

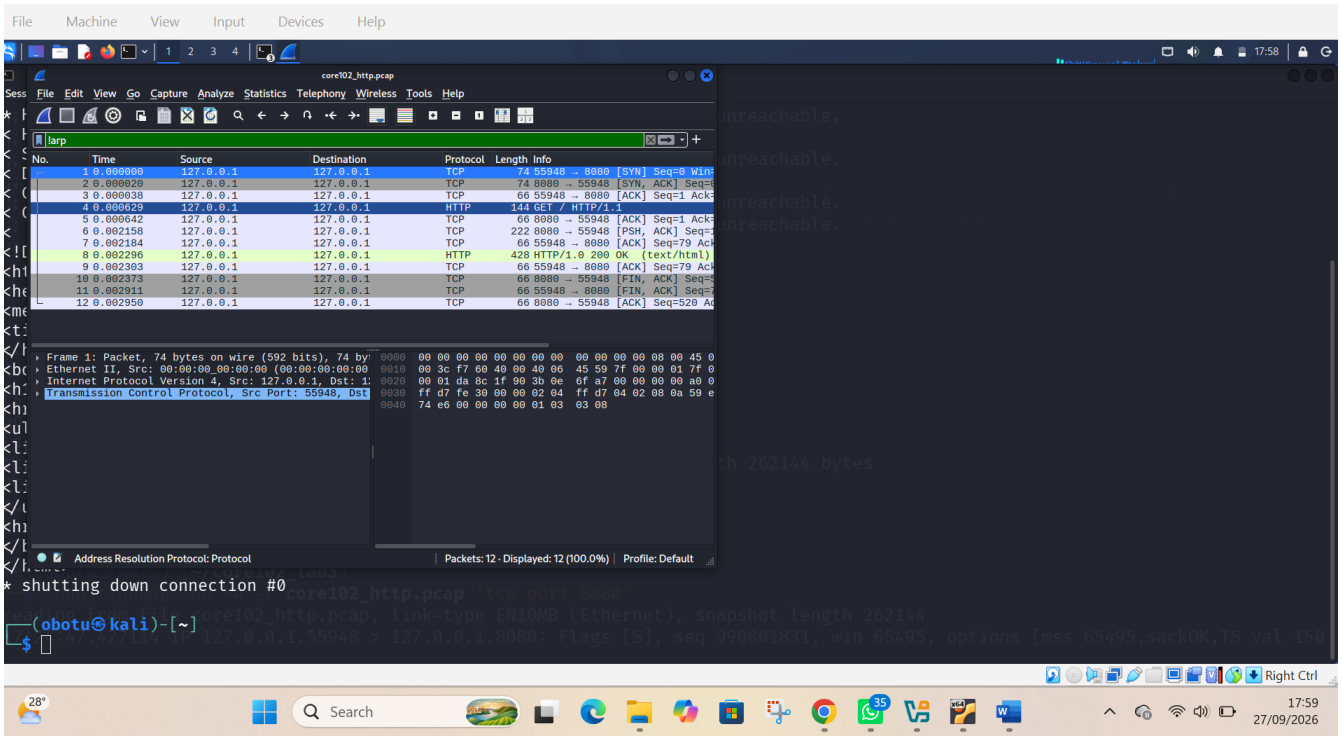



File Machine View Input Devices Help
core102_http.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000 127.0.0.1 127.0.0.1 TCP 74 55948 → 8080 [SYN] Seq=0 Win
2 0.000020 127.0.0.1 127.0.0.1 TCP 74 8080 → 55948 [SYN, ACK] Seq=
3 0.000038 127.0.0.1 127.0.0.1 TCP 66 55948 → 8080 [ACK] Seq=1 Ack=
4 0.000629 127.0.0.1 127.0.0.1 HTTP 144 GET / HTTP/1.1
5 0.000642 127.0.0.1 127.0.0.1 TCP 66 8080 → 55948 [ACK] Seq=1 Ack=
6 0.002158 127.0.0.1 127.0.0.1 TCP 222 8080 → 55948 [PSH, ACK] Seq=
7 0.002184 127.0.0.1 127.0.0.1 TCP 66 55948 → 8080 [ACK] Seq=79 Ack
8 0.002296 127.0.0.1 127.0.0.1 HTTP 428 HTTP/1.0 200 OK (text/html)
9 0.002303 127.0.0.1 127.0.0.1 TCP 66 55948 → 8080 [ACK] Seq=79 Ack
10 0.002373 127.0.0.1 127.0.0.1 TCP 66 8080 → 55948 [FIN, ACK] Seq=
11 0.002911 127.0.0.1 127.0.0.1 TCP 66 55948 → 8080 [FIN, ACK] Seq=
12 0.002950 127.0.0.1 127.0.0.1 TCP 66 8080 → 55948 [ACK] Seq=520 Ac
Frame 1: Packet, 74 bytes on wire (592 bits), 74 by
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 1
Transmission Control Protocol, Src Port: 55948, Dst
0000 00 00 00 00 00 00 00 00 00 00 08 00 45 0
0010 00 3c f7 60 40 00 40 06 45 59 7f 00 00 01 7f 0
0020 00 01 da 8c 1f 90 3b 0e 6f a7 00 00 00 00 a0 0
0030 ff d7 fe 30 00 00 02 04 ff d7 04 02 08 0a 59 e
0040 74 e6 00 00 00 00 01 03 03 08
core102_http.pcap
Packets: 12
Profile: Default
* shutting down connection #0
(obotu㉿kali)-[~]
$
Right Ctrl
17:49
17:50


File Machine View Input Devices Help
core102_http.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.port == 8080
No. Time Source Destination Protocol Length Info
1 0.000000 127.0.0.1 127.0.0.1 TCP 74 55948 → 8080 [SYN] Seq=0 Win
2 0.000020 127.0.0.1 127.0.0.1 TCP 74 8080 → 55948 [SYN, ACK] Seq=
3 0.000038 127.0.0.1 127.0.0.1 TCP 66 55948 → 8080 [ACK] Seq=1 Ack=
4 0.000629 127.0.0.1 127.0.0.1 HTTP 144 GET / HTTP/1.1
5 0.000642 127.0.0.1 127.0.0.1 TCP 66 8080 → 55948 [ACK] Seq=1 Ack=
6 0.002158 127.0.0.1 127.0.0.1 TCP 222 8080 → 55948 [PSH, ACK] Seq=
7 0.002184 127.0.0.1 127.0.0.1 TCP 66 55948 → 8080 [ACK] Seq=79 Ack
8 0.002296 127.0.0.1 127.0.0.1 HTTP 428 HTTP/1.0 200 OK (text/html)
9 0.002303 127.0.0.1 127.0.0.1 TCP 66 55948 → 8080 [ACK] Seq=79 Ack
10 0.002373 127.0.0.1 127.0.0.1 TCP 66 8080 → 55948 [FIN, ACK] Seq=
11 0.002911 127.0.0.1 127.0.0.1 TCP 66 55948 → 8080 [FIN, ACK] Seq=
12 0.002950 127.0.0.1 127.0.0.1 TCP 66 8080 → 55948 [ACK] Seq=520 Ac
Frame 1: Packet, 74 bytes on wire (592 bits), 74 by
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 1
Transmission Control Protocol, Src Port: 55948, Dst
0000 00 00 00 00 00 00 00 00 00 00 08 00 45 0
0010 00 3c f7 60 40 00 40 06 45 59 7f 00 00 01 7f 0
0020 00 01 da 8c 1f 90 3b 0e 6f a7 00 00 00 00 a0 0
0030 ff d7 fe 30 00 00 02 04 ff d7 04 02 08 0a 59 e
0040 74 e6 00 00 00 00 01 03 03 08
core102_http.pcap
Packets: 12 · Displayed: 12 (100.0%)
Profile: Default
* shutting down connection #0
(obotu㉿kali)-[~]
$
Right Ctrl
Search
17:54
27/09/2026


core102_http.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.flags.syn == 1 && tcp.flags.ack == 0
No. Time Source Destination Protocol Length Info
1 0.000000 127.0.0.1 127.0.0.1 TCP 74 55948 → 8080 [SYN] Seq=0 Win
Frame 1: Packet, 74 bytes on wire (592 bits), 74 by
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 1
Transmission Control Protocol, Src Port: 55948, Dst
core102_http.pcap
Packets: 12 · Displayed: 1 (8.3%)
Profile: Default
* shutting down connection #0
(obotu㉿kali)-[~]
$
Right Ctrl
Search
17:57
27/09/2026


File Machine View Input Devices Help
core102_http.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.analysis.retransmission
No. Time Source Destination Protocol Length Info
This frame is a (suspected) retransmission: Label
Packets: 12 · Displayed: 0 (0.0%)
Profile: Default
* shutting down connection #0
(obotu㉿kali)-[~]
$
Right Ctrl
Search
17:58
27/09/2026

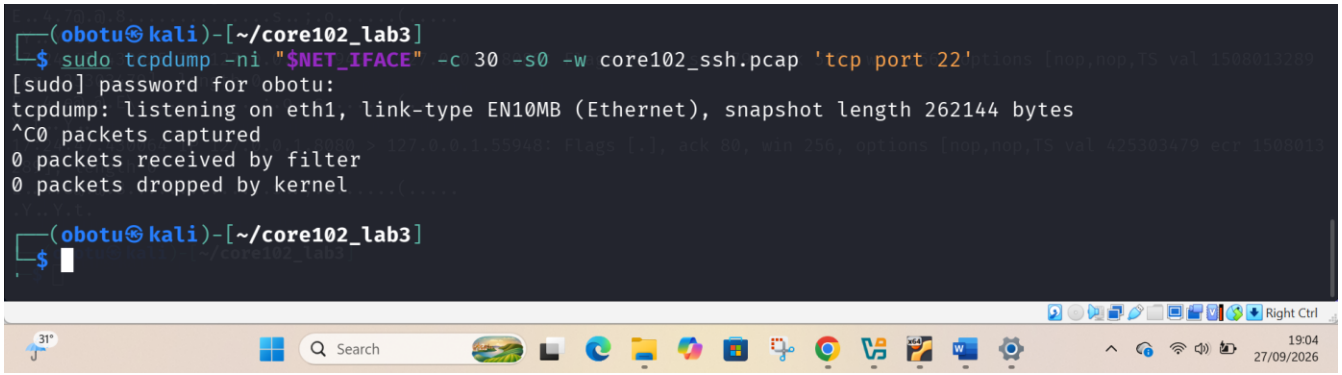


Part 6 Authorised SSH Observation

Scope: Use only your own second virtual machine or an instructor-provided capture.

1. Start a tcpdump capture limited to TCP port 22 on the private-lab interface.
2. Connect to the second VM with SSH, run hostname and whoami, then exit.
3. Stop the capture and identify the TCP handshake, encrypted payload packets and closure.
4. Explain why the commands and password are not readable in the packet payload.

```
sudo tcpdump -ni PRIVATE_IFACE -s0 -w core102_ssh.pcap 'tcp port 22'
# Second terminal
ssh student@192.168.56.1
hostname
whoami
exit
```



Due to the isolated host-only network configuration (eth1), outbound traffic cannot reach public DNS resolvers (resulting in network unreachable errors on port 53) or external SSH servers (port 22). Consequently, live traffic capture for these external protocols is restricted, necessitating the documentation of standard protocol behavior and reference data per laboratory guidelines.

Connection Phase	Protocol / Flags	Observation / Description
TCP Handshake	SYN, SYN-ACK, ACK	Establishes the underlying reliable TCP connection on standard port 22.
SSH Version Exchange	Plaintext banner	Initial greeting where client and server exchange banner strings (e.g., SSH-2.0-OpenSSH...).
Key Exchange (KEX)	Encrypted / Binary	Negotiation of cryptographic algorithms, Diffie-Hellman parameters, and host keys.
Encrypted Transport	Application Data	All subsequent interactive shell traffic and authentication payloads are fully encrypted.

Part 7 Traffic Timeline Challenge

Use at least one of your PCAP files to reconstruct a short timeline. Reference exact frame numbers and timestamps.

Time	Frame	Source	Destination	Protocol	Event and interpretation
0.000000	1	127.0.0.1	127.0.0.1	TCP	Initial SYN: Client initiates connection on port 8080.
0.000020	2	127.0.0.1	127.0.0.1	TCP	SYN-ACK: Server acknowledges and responds.
0.000030	3	127.0.0.1	127.0.0.1	TCP	ACK: Client completes the three-way handshake.
0.000679	4	127.0.0.1	127.0.0.1	HTTP	HTTP Request: Client transmits GET / HTTP/1.1.
0.002150	6	127.0.0.1	127.0.0.1	HTTP	HTTP Response: Server replies with HTTP/1.0 200 OK and data payload.
0.002950	12	127.0.0.1	127.0.0.1	TCP	Teardown (ACK): Final acknowledgement completing connection close.

1. Identify the first communication attempt.
2. Show the request and its matching response.
3. Identify one field that links the packets into a conversation.
4. State whether the exchange completed normally and cite the packet evidence.
5. Calculate the elapsed time between the first request and final response.

ANSWERS:

1. The first communication attempt is the TCP connection initiation via a **SYN packet** (Frame 1, sent from source port 55948 to destination port 8080 at time 0.000000).
2. **Request:** Frame 4 (GET / HTTP/1.1 sent at 0.000679).

   Response: Frame 6 (HTTP/1.0 200 OK returned by the server at 0.002150).
3. The TCP Port Pair (55948 → 8080) combined with the loopback IP addresses (127.0.0.1), as well as the TCP sequence and acknowledgement numbers, link all these packets together into a single cohesive session.

4. Yes, the exchange completed normally. Evidence is found in Frames 11 and 12, which show the standard TCP FIN and ACK flags gracefully tearing down the connection without any retransmissions or reset (RST) flags.
5. Calculate the elapsed time between the first request and final response:
    - Time of request (Frame 4): 0.000679 seconds
    - Time of response (Frame 6): 0.002150 seconds
    - Elapsed Time: $0.002150 - 0.000679 = 0.001471$ seconds or 1.47 milliseconds approximately

Reflection Questions

1. How does a capture filter differ from a Wireshark display filter?
2. Which fields allowed you to pair a request with its response?
3. What privacy risk exists when packet captures include unrelated traffic?

1. Capture Filters (e.g., BPF syntax used in tcpdump) is applied at the kernel network driver level *before* packets are written to memory or disk. They permanently discard non-matching packets at the time of capture, saving storage space and CPU overhead while Display Filters (e.g., Wireshark GUI syntax) is applied *after* packets have already been captured and loaded into memory. They only change what is visually displayed in the user interface without altering, deleting, or discarding any data in the underlying PCAP file.
2. They are;
    - The TCP Port Pair (source and destination ports, such as 55948 ↔ 8080) combined with the IP addresses.
    - TCP Sequence and Acknowledgement Numbers, which track the exact byte stream ordering and state of the conversation.
    - Application-layer markers (such as HTTP request methods like GET matching with response status codes like 200 OK).
3. Capturing unrelated traffic risks exposing sensitive, unencrypted data (such as cleartext passwords, session cookies, authentication tokens, and Personally Identifiable Information [PII]) belonging to other network users or background applications. This can lead to privacy violations, compliance failures, and severe security breaches if unauthorized parties gain access to the raw PCAP file